

CLUTCH BACKEND · IMPLEMENTATION REPORT

쿠폰 성공 수량 집계 안정성 개선 보고서

상태 계약 · 계산형 만료 · 테스트 격리 · 운영 메트릭 · 다중 인스턴스 제어

작성일 2026-08-26 (Asia/Seoul)

최종 재검증 2026-08-27 (Asia/Seoul)

작업 브랜치 feat/CLUTCH-268

기준 dev 5b81ad0

검증 Gradle clean build · 507 tests passed

제외 범위 k6 스크립트는 요청에 따라 수정하지 않음

1. 결론

쿠폰 발급의 핵심 Redis·MySQL 흐름은 유지하면서, 기존 데이터 조회 실패 가능성, 반복 테스트 충돌, 운영 가시성 부족, 다중 서버 중복 집계 문제를 개선했다. 데이터베이스 스키마는 이미 필요한 상태를 허용하므로 새 migration은 만들지 않았다.

상태 계약 일치

DB에 존재하는 요청 `CANCELLED` 와 쿠폰 `EXPIRED` 를 Java enum, JPA, 관리자 API가 안전하게 처리한다.

반복 가능한 테스트

테스트 claim ID 충돌을 방지하고 테스트 커넥션 풀과 자동 집계를 격리했다.

운영 메트릭

집계 시간, 실패, 락 스킵, 비교 항목, 갱신 항목을 Prometheus에서 확인한다.

다중 인스턴스 안전성

MySQL named lock을 즉시 획득한 한 인스턴스만 전체 집계를 실행한다.

2. 변경 전 확인된 문제

영역	문제	위험
상태 계약	DB는 요청 <code>CANCELLED</code> , 쿠폰 <code>EXPIRED</code> 를 허용하지만 Java enum에 없음	JPA 조회·관리자 응답 변환 중 예외 가능
테스트 데이터	<code>claimId=2001/3001/8001</code> 을 반복 사용	공유 로컬 DB의 기존 데이터와 충돌하여 clean build 실패
운영 관측	집계 성공 여부와 실행 시간을 알 수 없음	지연·실패·불필요한 반복 실행을 탐지하기 어려움
서버 확장	<code>@Scheduled</code> 가 인스턴스마다 동일 전체 집계를 실행	서버 수에 비례해 MySQL 전체 집계 부하 증가

3. 상태 계약 개선

3.1 정확히 바꾼 내용

- `ClaimRequestStatus` 에 `CANCELLED` 추가
- `UserCouponStatus` 에 `EXPIRED` 추가
- 관리자 발급 내역의 문자열 변환이 두 상태를 정상 응답하도록 테스트 추가
- JPA가 DB의 `CANCELLED` 요청과 `EXPIRED` 쿠폰을 역직렬화하는 통합 테스트 추가
- `ISSUED` 쿠폰은 기준 시각이 `expiresAt` 이상이면 사용자·관리자 응답에서 `EXPIRED` 로 계산
- `ISSUED` / `EXPIRED` 필터도 저장 상태와 만료 시각을 함께 사용하도록 통일
- 쿠폰 사용과 관리자 취소를 모두 만료 시각 이전의 `ISSUED` 상태에서에서만 조건부 갱신

범위 통제: 발급 요청 `CANCELLED` 는 기존 DB 값을 안전하게 읽는 호환성 범위로 유지한다. 쿠폰 `EXPIRED` 는 배치로 대량 갱신하지 않고 `expiresAt` 을 기준으로 계산해 배치 지연 없이 같은 결과를 반환한다.

3.2 상태 계약

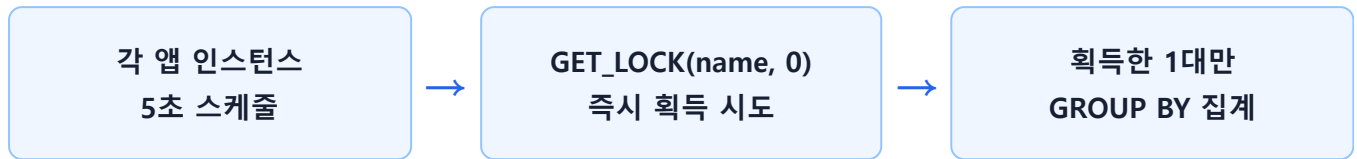
대상	허용 상태	현재 생성 경로
coupon_claim_request	PENDING, SUCCEEDED, FAILED, CANCELLED	PENDING → SUCCEEDED 또는 FAILED
user_coupon	ISSUED, USED, EXPIRED, CANCELLED	ISSUED → USED 또는 CANCELLED, 만료 시각부터 유효 상태 EXPIRED

4. 테스트 격리 개선

- `UserCouponRepositoryTest` 는 최신 `dev` 의 충돌 방지용 `CLAIM_BASE` 대역을 유지
- `CouponIssuanceServiceFailureTest` 의 claim ID를 UUID 기반으로 생성
- 테스트 설정에서 성공 수량 자동 스케줄러를 비활성화하고 테스트가 필요한 경우 서비스를 직접 호출
- 일반 테스트 Hikari 풀을 4개로 제한해 Spring context 누적으로 MySQL 151 연결 한도를 넘지 않도록 조정
- 실제 동시성 테스트는 기존 클래스 설정의 30개 풀을 유지

첫 전체 실행에서 신규 테스트 context가 추가되면서 MySQL `Too many connections` 가 재현됐다. 이를 숨기지 않고 테스트 전용 풀 제한을 적용한 뒤 전체 빌드를 다시 실행해 통과시켰다.

5. 다중 인스턴스 집계 제어



- 락 이름: `clutch:coupon-success-count-synchronization`
- 공유 MySQL 서버에서는 `COUPON_SUCCESS_COUNT_SYNC_LOCK_NAME` 으로 환경별 이름 지정
- 대기 시간: 0초. 다른 인스턴스가 실행 중이면 해당 주기를 즉시 건너뛴다
- 정상·예외 종료 모두 `RELEASE_LOCK` 실행
- DB 연결이 물리적으로 종료되어도 MySQL이 named lock을 자동 해제
- 락 보유 중 실제 집계는 기존 `@Transactional` 서비스에서 수행

스케줄 책임을 `CouponSuccessCountSynchronizationScheduler` 로 분리하고, 실제 데이터 보정은 `CouponSuccessCountSynchronizer` 가 담당한다. 집계 서비스는 비교 항목 수와 갱신 항목 수를 결과 객체로 반환한다.

5.1 잠금 검증

- 한 DB 연결이 락을 가진 동안 다른 연결의 작업이 실행되지 않음을 통합 테스트로 확인
- 작업 종료 후 다음 실행이 동일 락을 다시 획득함을 확인
- 스케줄러 단위 테스트에서 성공·스킵·실패 세 경로를 모두 확인

6. 추가한 운영 메트릭

Prometheus 이름	의미
<code>clutch_coupon_success_count_synchronization_duration_seconds</code>	outcome(success/skipped/failure)별 실행 시간
<code>clutch_coupon_success_count_synchronization_failures_total</code>	집계 예외 누적 횟수
<code>clutch_coupon_success_count_synchronization_lock_skipped_total</code>	다른 인스턴스 실행 때문에 건너뛴 누적 횟수
<code>clutch_coupon_success_count_synchronization_items_scanned</code>	실행당 비교한 이벤트 항목 수
<code>clutch_coupon_success_count_synchronization_items_updated</code>	실행당 실제 갱신한 이벤트 항목 수

7. 검증 결과

7.1 자동화 테스트

검증	결과
상태 정책 관련 테스트 6개 클래스	16 tests · 0 failures
<code>gradlew.bat clean build</code>	507 tests · 0 failures · 0 errors · 0 skipped
컴파일·bootJar·check·build	BUILD SUCCESSFUL (1m 57s)
Flyway / Hibernate validate	MySQL 8.4에서 통과

7.2 실행 중 메트릭 확인

최신 이미지를 로컬 Docker Compose에 기동하고 `/actuator/prometheus` 에서 확인했다.

관측값	결과
성공 실행 수	관측 시점 6회
실행 시간 합계 / 최대	137.96 ms / 64.13 ms
실행당 비교 항목	8개
실제 갱신 항목	0개 — 이미 정합한 상태
실패 / 락 스킵	0 / 0

7.3 관련 테스트 구성

- 상태 JPA 호환: `CouponClaimRequestRepositoryTest`, `UserCouponRepositoryTest`
- 계산형 만료 경계: `UserCouponTest`, `CouponQueryServiceTest`
- 만료 후 취소 차단: `AdminCouponServiceTest`, `UserCouponRepositoryTest`
- 유효 상태 필터: `UserCouponRepositoryTest`, `AdminCouponClaimQueryRepositoryTest`
- 관리자 응답 호환: `AdminCouponClaimServiceTest`
- 집계 결과: `CouponSuccessCountSynchronizerTest`
- 메트릭 세 경로: `CouponSuccessCountSynchronizationSchedulerTest`
- 실제 MySQL 락 경쟁·해제: `MySqlCouponSuccessCountSynchronizationLockTest`
- Outbox 격리: `CouponIssuanceServiceFailureTest`

8. 변경 파일

파일	변경 내용
ClaimRequestStatus.java	CANCELLED 조회 호환 상태 추가
UserCouponStatus.java	EXPIRED 조회 호환 상태 추가
UserCoupon.java / CouponResponse.java	저장 상태와 만료 시각을 반영한 유효 상태 계산
UserCouponRepository.java	유효 상태 목록 필터와 만료 후 취소 차단
CouponQueryService.java	한 기준 시각으로 목록·단건 응답 상태 계산
CouponUseService.java / AdminCouponService.java	사용·취소 실패 사유를 동일한 유효 상태로 판정
AdminCouponClaimQueryRepository.java	관리자 응답·필터에 계산형 EXPIRED 적용
CouponSuccessCountSynchronizer.java	스케줄 책임 분리, 처리 결과 반환
CouponSuccessCountSynchronizationResult.java	비교·갱신 항목 수 결과 계약
CouponSuccessCountSynchronizationScheduler.java	스케줄, 잠금, Micrometer 메트릭
CouponSuccessCountSynchronizationLock.java	중복 실행 제어 인터페이스
MySQLCouponSuccessCountSynchronizationLock.java	GET_LOCK/RELEASE_LOCK 구현
application.example.yaml	COUPON_SUCCESS_COUNT_SYNC_ENABLED 설정 추가
src/test/resources/application.properties	자동 집계 비활성화, 테스트 풀 4개 제한
관련 테스트	상태 경계·필터·취소, 잠금, 메트릭, 고정 ID 충돌 검증
docs/domain/coupon.md	상태 계약과 단일 집계 규칙 확정
docs/verification/coupon-integrity.sql / README.md	현재 enum 계약 반영, 계산형 만료 건수를 INFO로 분류
docs/adr/003-async-coupon-success-count.md	락·메트릭 결정과 결과 기록
쿠폰 정합성 결과 문서	기존 WARN 후속 조치 완료 기록

9. 운영 적용과 확인

- 기본값은 `COUPON_SUCCESS_COUNT_SYNC_ENABLED=true` 이며 5초마다 실행한다.
- 같은 MySQL 서버를 여러 환경이 공유하면 lock name을 환경별로 다르게 설정한다.

3. 긴급 비활성화가 필요하면 환경변수를 false로 설정한다.
4. Grafana에서 duration max, failures total, lock skipped total을 우선 대시보드에 배치한다.
5. 다중 인스턴스 배포 후 한 인스턴스의 success와 나머지 인스턴스의 skipped 증가를 확인한다.
6. failure 증가 시 MySQL 연결, GROUP BY 실행 계획, 락 연결 상태를 함께 확인한다.

판정: 요청한 k6 외 개선 범위는 구현과 자동화 검증을 완료했다. 발급 허용 여부는 계속 Redis가 결정하며, 이번 변경은 그 핵심 경로의 의미를 바꾸지 않는다.

이 보고서는 저장소의 실제 diff, Gradle XML 테스트 결과, 실행 중 Prometheus 응답을 기준으로 작성했다.